DEVOPS COURSE · SESSION 21 CAPSTONE

Kirana Ledger 

A credit book for a small Indian grocery shop, taken from a laptop to a monitored, auto-scaling Kubernetes cluster.

Milap Kothari · github.com/milap1afk/devops-homework

FastAPI · PostgreSQL + Alembic · Vite + Nginx · Docker · GitHub Actions · Trivy · Semgrep · Gitleaks · Terraform (VPC + EKS) · Helm · Ingress · HPA · Prometheus · Grafana · Argo CD

THE PROBLEM

Every kirana shop runs on *udhar*

Neighbours take groceries on credit and pay back later. Today that lives in a paper notebook.

उधार udhar

Credit given

Balance goes **up**

वापस payment

Credit paid back

Balance goes **down**

बिक्री sale

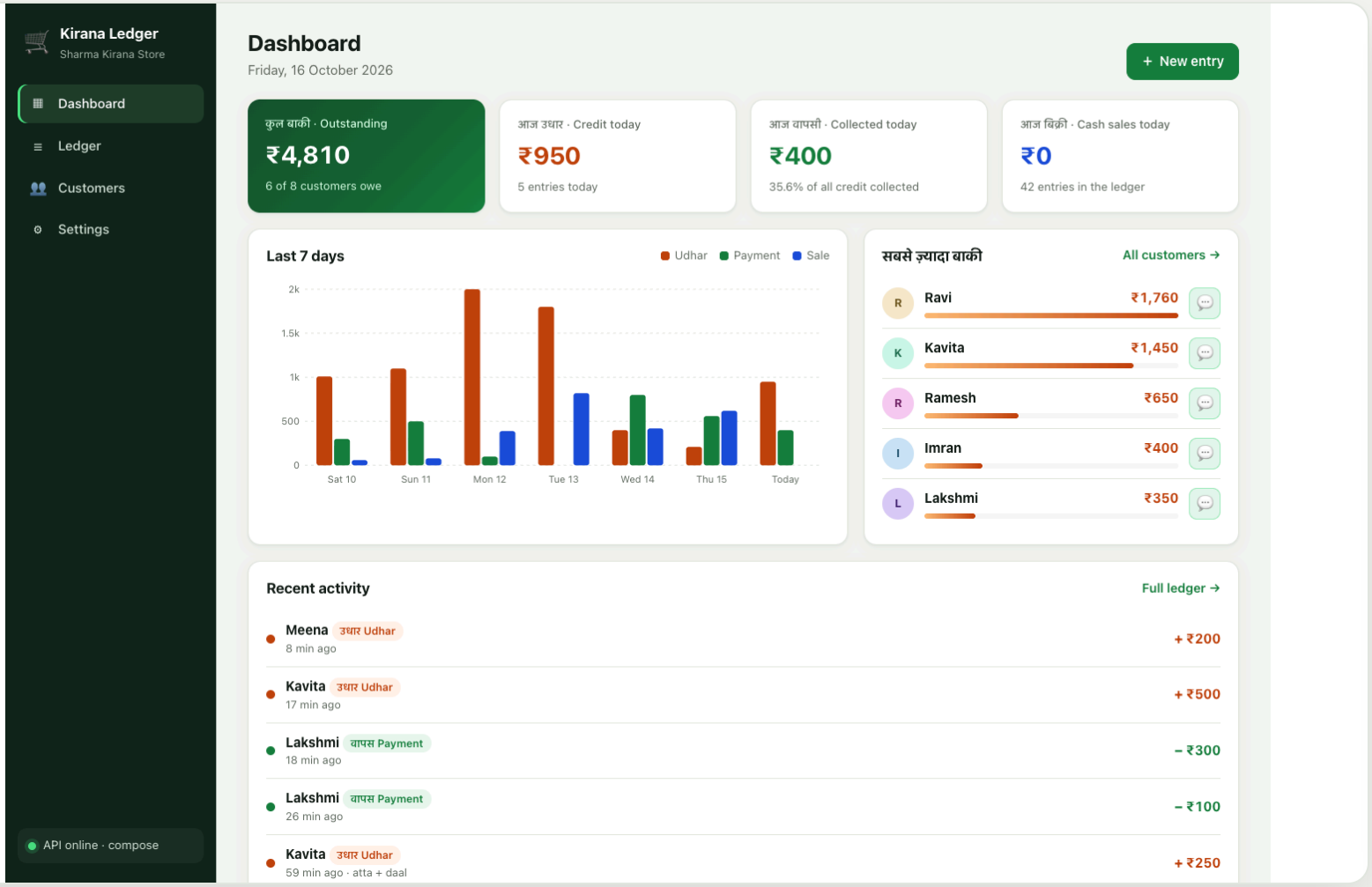
Cash sale

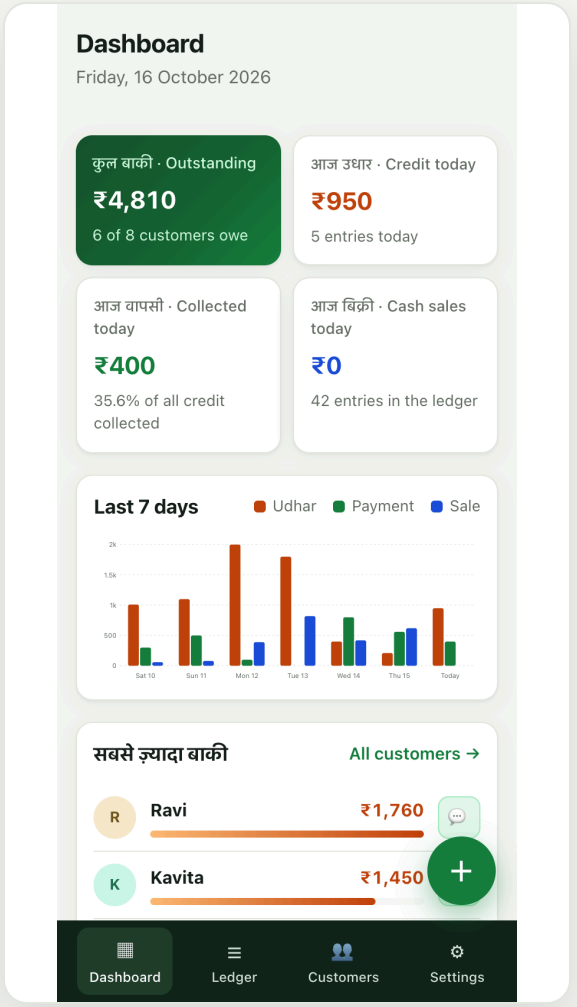
Counted in today's takings

- **कौन कितना बाकी** — who owes how much, biggest first
- "Ramesh bhai", "Rameshbhai", "Shri Ramesh ji" → one customer **Ramesh**
- One-tap WhatsApp reminder: *"Ramesh ji, ₹300 बाकी हैं."*

THE APPLICATION · V2.1.0

A real dashboard, not a CRUD page

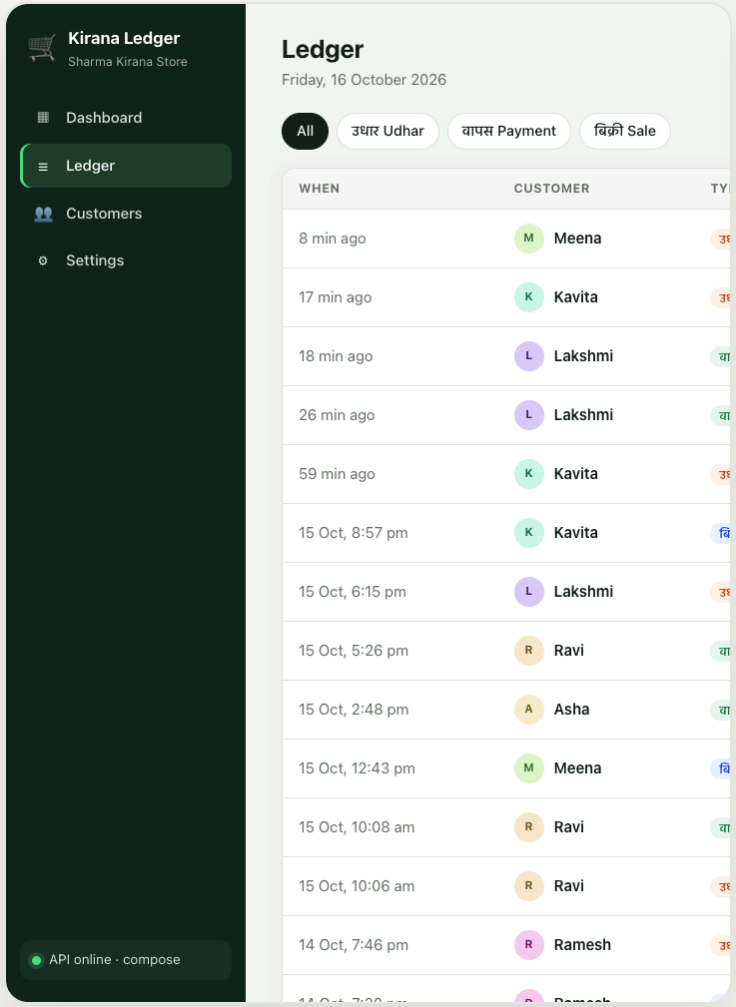


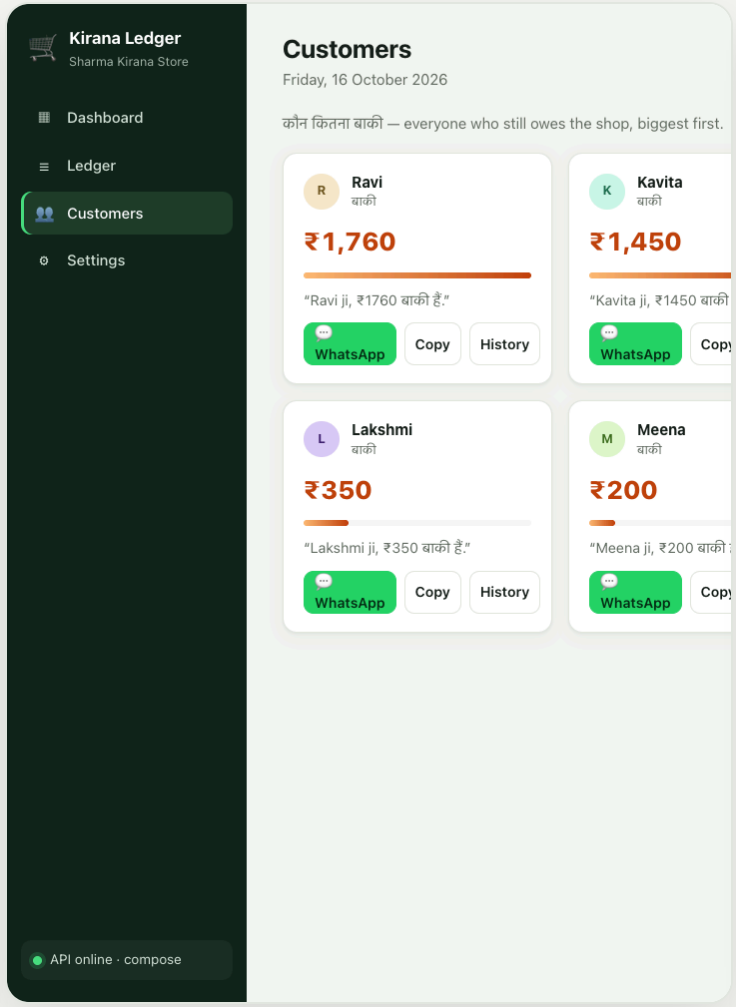


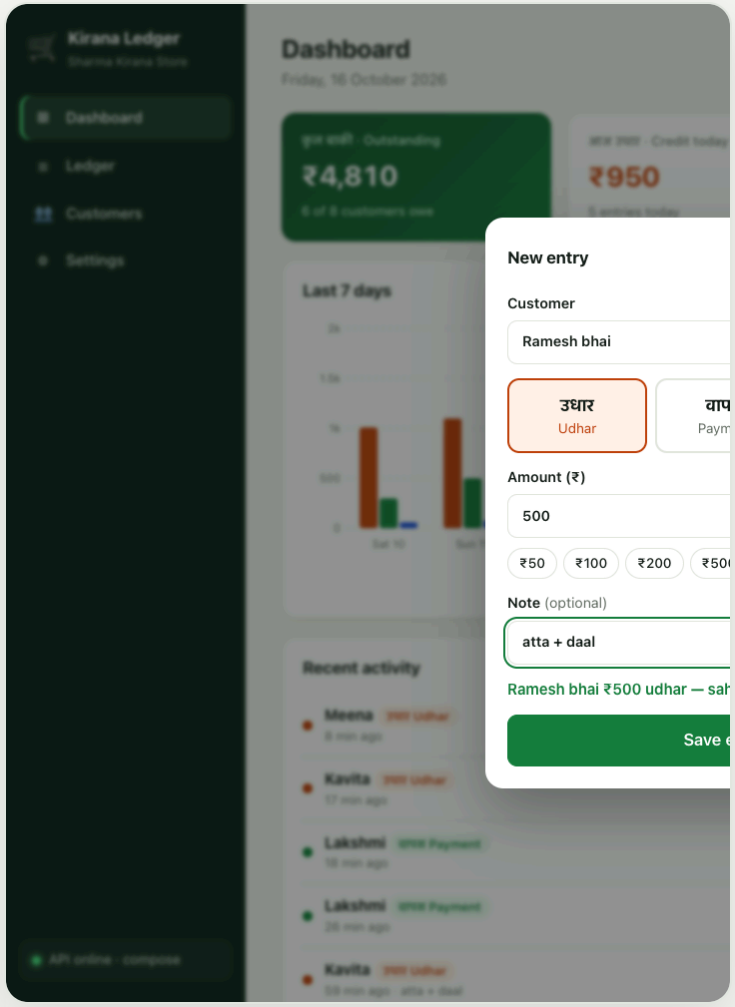
KPIs · 7-day chart · top dues · activity feed · phone layout · dark mode

THE APPLICATION

Ledger, customers and the entry flow



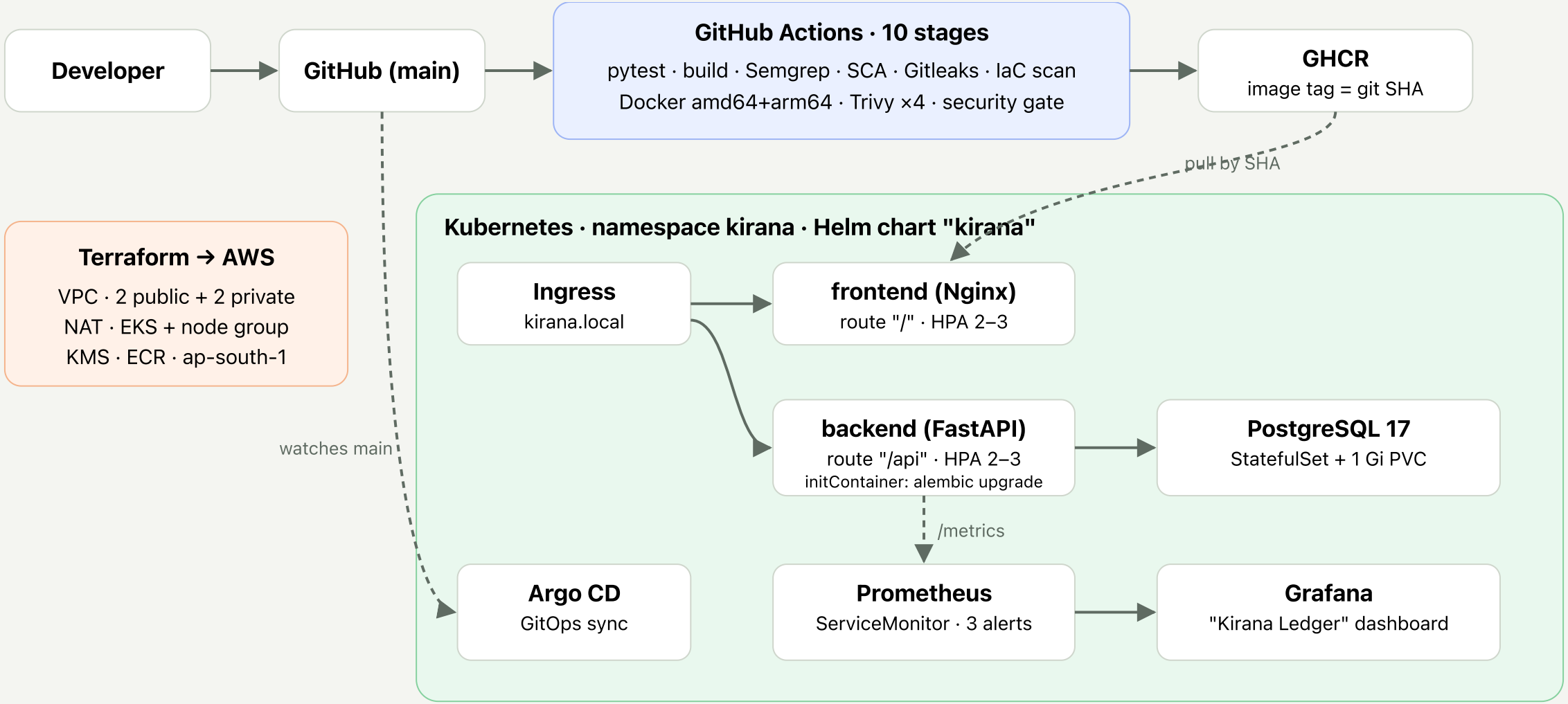




Filter by type, search, edit/delete · running balance history · "Ramesh bhai ₹500 udhar — sahi hai?"

ARCHITECTURE

From git push to a monitored cluster



Same chart runs on minikube (Argo CD), kind (CI deploy) and EKS (Terraform)

BACKEND · M1

FastAPI + SQLAlchemy + Alembic

METHOD	PATH	PURPOSE
GET	`/health`	liveness — never touches the DB
GET	`/ready`	readiness — `SELECT 1`, else 503
GET	`/metrics`	Prometheus + `kirana_ledger_entries_total`
GET	`/api/stats`	KPIs + 7-day trend (IST)
GET	`/api/entries`	`?type= ?q= ?customer= ?limit=`
POST · PUT · DELETE	`/api/entries/{id}`	writes need `X-API-Key`
GET	`/api/customers[/{name}]`	balances + reminder text

Design choices

- 12 endpoints; Swagger at `/docs`
- Schema owned by Alembic migration `0001`, with CHECK constraints (`amount > 0`, valid type)
- Migrations run **once** (Compose one-shot service / K8s initContainer), not in every API process
- Names normalised in the Pydantic schema, so every path stores "Ramesh"
- Config only from env: ConfigMap + Secret

application/backend/app/main.py

TESTING · M2

21 tests, 92 % coverage, first quality gate

pytest -v (21 passed, 92% coverage)

```
$ flake8 . && pytest -v --cov=app --cov-fail-under=80
============================= test session starts ==============================
collecting ... collected 21 items

tests/test_api.py::test_clean_name_strips_honorifics[Ramesh] PASSED
tests/test_api.py::test_clean_name_strips_honorifics[Ramesh bhai] PASSED
tests/test_api.py::test_clean_name_strips_honorifics[Rameshbhai] PASSED
tests/test_api.py::test_clean_name_strips_honorifics[Shri Ramesh ji] PASSED
tests/test_api.py::test_clean_name_strips_honorifics[  ramesh  ] PASSED
tests/test_api.py::test_health PASSED
tests/test_api.py::test_ready PASSED
tests/test_api.py::test_create_and_list_entries PASSED
tests/test_api.py::test_get_update_delete_entry PASSED
tests/test_api.py::test_balance_and_reminder PASSED
tests/test_api.py::test_customers_sorted_biggest_first_and_settled_hidden PASSED
tests/test_api.py::test_rejects_invalid_entries[bad0] PASSED
tests/test_api.py::test_rejects_invalid_entries[bad1] PASSED
tests/test_api.py::test_rejects_invalid_entries[bad2] PASSED
tests/test_api.py::test_unknown_entry_is_404 PASSED
tests/test_api.py::test_api_key_protects_writes PASSED
tests/test_api.py::test_metrics_endpoint PASSED
tests/test_api.py::test_filter_entries_by_type_and_search PASSED
tests/test_api.py::test_stats_today_outstanding_and_trend PASSED
tests/test_api.py::test_stats_empty_ledger PASSED
tests/test_api.py::test_seed_builds_a_week_of_demo_data PASSED

---------- tests coverage ----------
_____________ coverage: platform linux, python 3.12.15-final-0 _____________

Name             Stmts   Miss  Cover
----------------------------------------
app/__init__.py      0      0   100%
app/config.py        8      0   100%
app/db.py           15      4    73%
app/main.py        110      6    95%
app/models.py       12      0   100%
app/schemas.py      51      1    98%
app/seed.py         42      9    79%
----------------------------------------
TOTAL              238     20    92%
Required test coverage of 80% reached. Total coverage: 91.60%
============================= 21 passed in 0.49s ===============================
```

tests

21 passed

coverage (CI fails below 80 %)

91.6 %

- Fresh SQLite DB per test via `dependency_overrides` — never production
- Covers CRUD, balances, filters, stats, API-key auth, 404/422, metrics, seed data
- `flake8` + `pytest` fail → no image is built

application/backend/tests/

Small, non-root images and a one-command stack

backend/Dockerfile

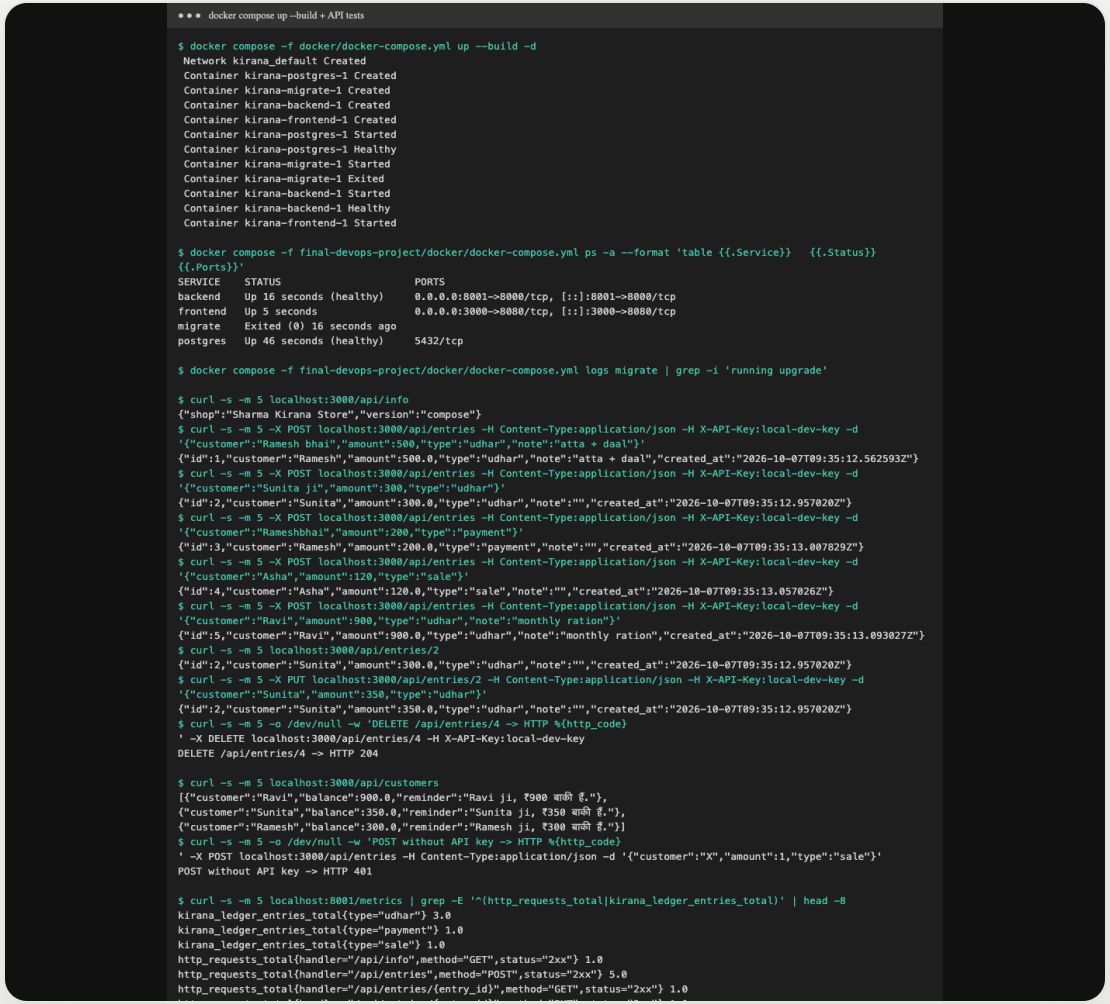
2 stages: deps → `python:3.12-slim` · `apt-get upgrade` · `USER 10001` · no DB needed to boot

frontend/Dockerfile

2 stages: `node:22-alpine` builds `dist/` → `nginx-unprivileged` on 8080 · `USER 101` · Node never ships

docker compose up --build

postgres **healthy** → migrate **exits 0** → backend **healthy** → frontend on `:3000`



Every push to main runs 10 stages

1 Test
flake8 · pytest · cov ≥ 80 · npm build

→

2–5 Scan code
Semgrep · pip-audit · npm audit · Gitleaks · Trivy config

→

6 Build
2 tiers × amd64 / arm64 native runners

→

7 Scan images
Trivy × 4

→

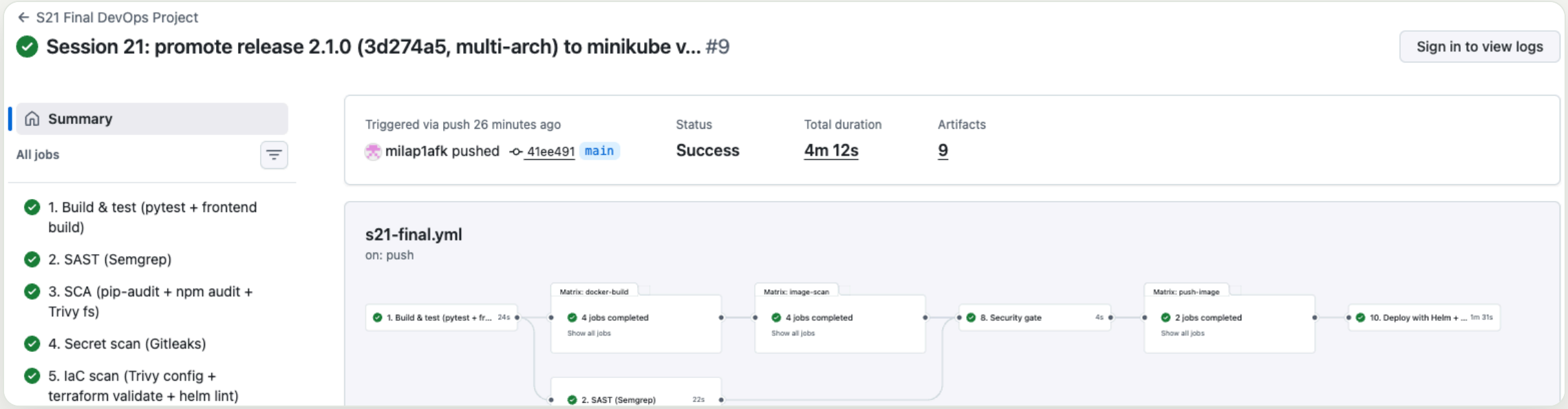
8 Gate
all scans green or stop

→

9 Push
GHCR multi-arch `:<sha>`

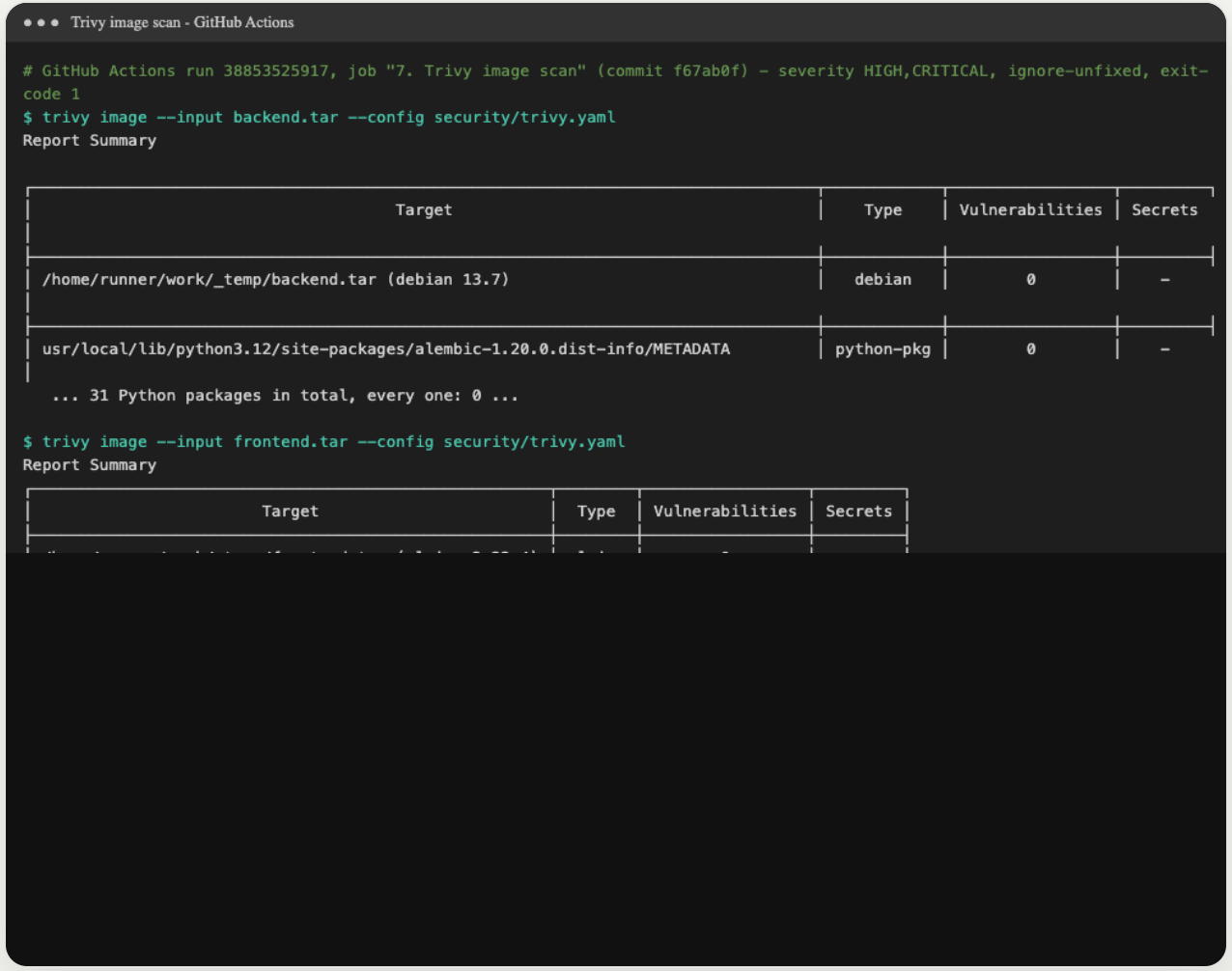
→

10 Deploy
Helm on kind + smoke test



.github/workflows/s21-final.yml · tag = commit SHA, never "latest"

Five scanners, one gate



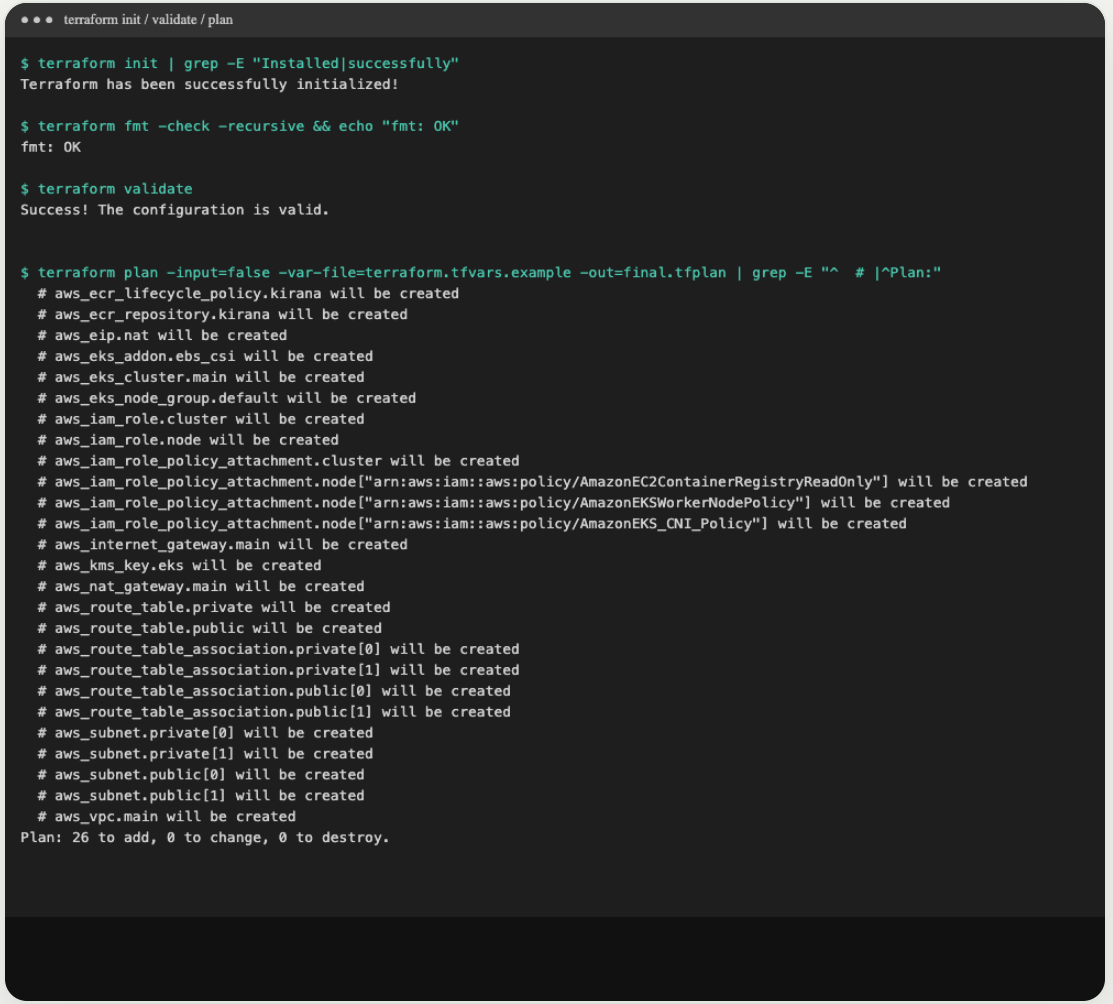
LAYER	TOOL
SAST	Semgrep (python, fastapi, js, nginx, docker, secrets)
Dependencies	pip-audit · npm audit · Trivy fs
Secrets	Gitleaks over full git history
IaC	Trivy config · terraform validate · helm lint
Images	Trivy, fixable HIGH/CRITICAL → fail

Result: 0 fixable HIGH/CRITICAL in all four images (Debian 13.7 + 31 Python packages; Alpine 3.23.4). "Clean" means nothing known *and fixable* today — so it runs on every push.

security/trivy.yaml · security/.gitleaks.toml

AWS infrastructure as code

- **network.tf** — VPC, 2 public + 2 private subnets in 2 AZs, IGW, NAT, route tables
- **eks.tf** — EKS cluster (secrets encrypted with KMS), managed node group `t3.medium` 2–4, EBS CSI add-on, IAM roles
- **ecr.tf** — image repo + lifecycle policy
- **terraform.tfvars.example** only — real vars are gitignored
- CI: `fmt -check` · `validate` · Trivy config scan



terraform/ · region ap-south-1 · Plan: 26 to add · always terraform destroy

One chart: Deployments, StatefulSet, Ingress, HPA

minikube: kirana namespace after the 2.1.0 rollout

```
$ kubectl -n argocd get application kirana-final
NAME           SYNC STATUS   HEALTH STATUS
kirana-final   Synced        Healthy

$ kubectl -n kirana get pods -o wide | grep -v loadgen | cut -c1-118
NAME                                READY   STATUS      RESTARTS   AGE     IP             NODE       NO
kirana-backend-5dfcf8499f-m46kb     1/1     Running     0          2m43s   10.244.0.129   minikube   <n
kirana-backend-5dfcf8499f-sffbb     1/1     Running     0          9m16s   10.244.0.128   minikube   <n
kirana-backend-5dfcf8499f-t8wzw     1/1     Running     0          11m     10.244.0.125   minikube   <n
kirana-frontend-6c6dc59b6c-hwt99    1/1     Running     0          10m     10.244.0.127   minikube   <n
kirana-frontend-6c6dc59b6c-tjrnz    1/1     Running     0          11m     10.244.0.126   minikube   <n
kirana-postgres-0                   1/1     Running     0          37m     10.244.0.119   minikube   <n

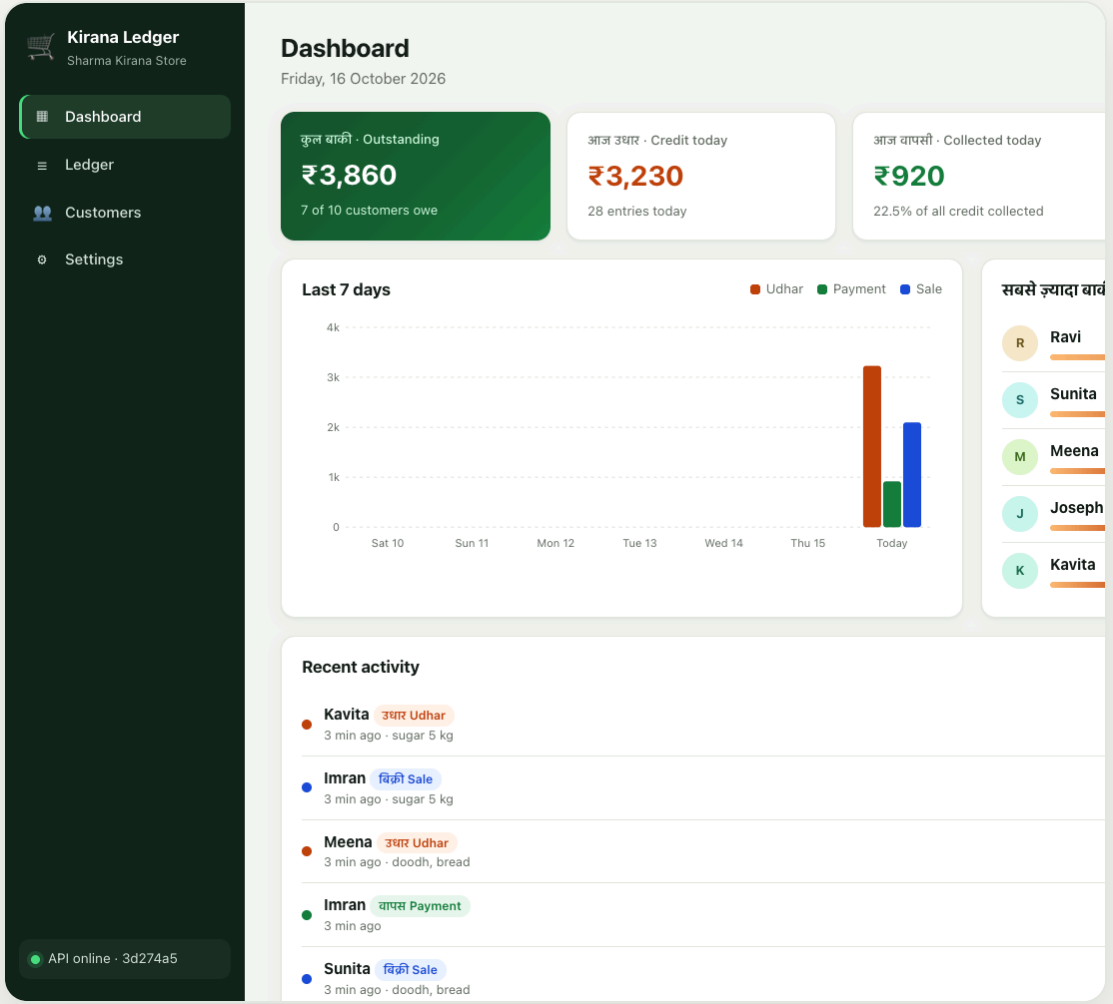
$ kubectl -n kirana get svc,ingress
NAME                       TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/kirana-backend     ClusterIP   10.99.150.51    <none>        80/TCP     8d
service/kirana-frontend    ClusterIP   10.100.57.234   <none>        80/TCP     8d
service/kirana-postgres    ClusterIP   None            <none>        5432/TCP   8d

NAME                                CLASS   HOSTS          ADDRESS        PORTS   AGE
ingress.networking.k8s.io/kirana    nginx   kirana.local   192.168.49.2   80      8d

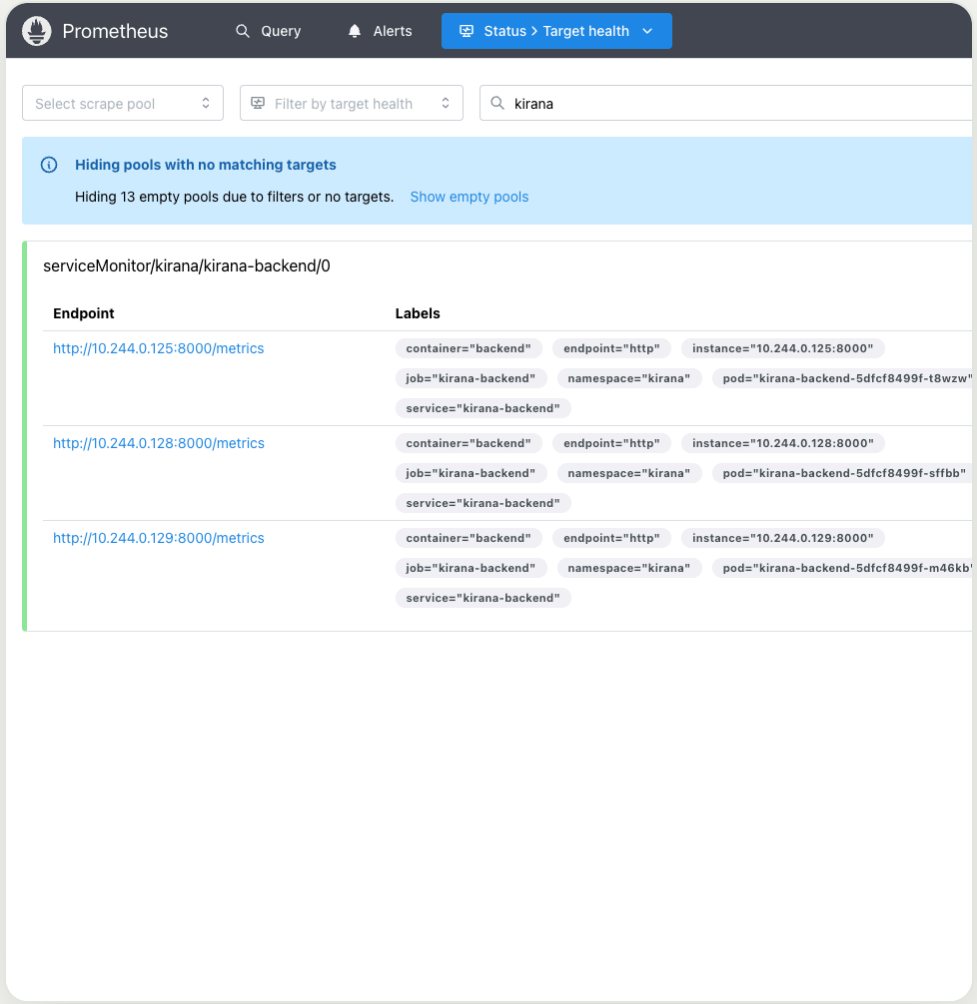
$ kubectl -n kirana get hpa
NAME              REFERENCE                    TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
kirana-backend    Deployment/kirana-backend    cpu: 145%/60%    2         3         3          8d
kirana-frontend   Deployment/kirana-frontend   cpu: 20%/70%     2         3         2          8d

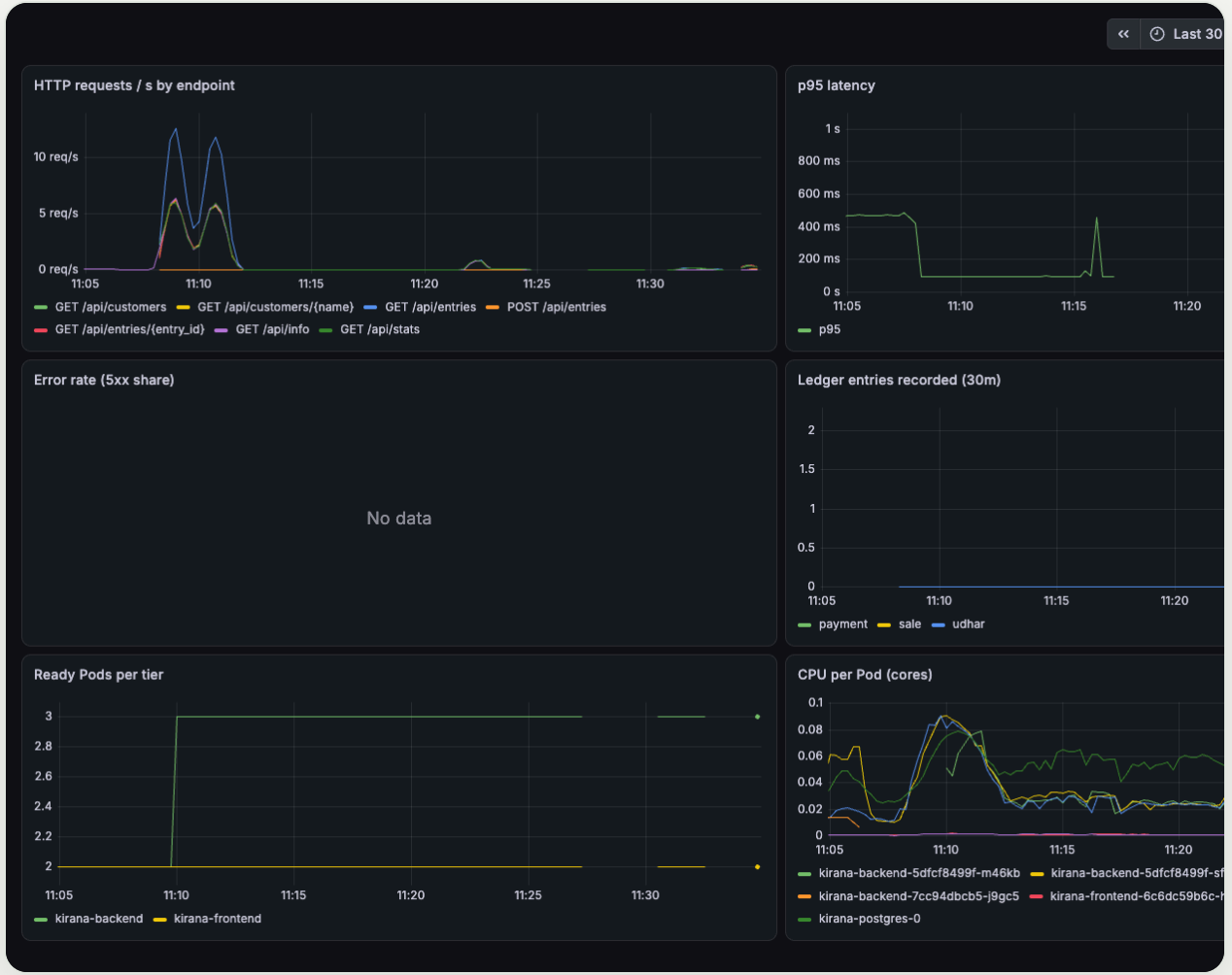
$ kubectl -n kirana get deploy -o 'custom-
columns=NAME:.metadata.name,READY:.status.readyReplicas,IMAGE:.spec.template.spec.containers[0].image'
NAME              READY   IMAGE
kirana-backend    3       ghcr.io/milap1afk/kirana-backend:3d274a5989723aebb67ee8bc3c9ba632b2c4947e
kirana-frontend   2       ghcr.io/milap1afk/kirana-frontend:3d274a5989723aebb67ee8bc3c9ba632b2c4947e

$ curl -s -H "Host: kirana.local" localhost:8089/api/info
{"shop":"Sharma Kirana Store","version":"3d274a5989723aebb67ee8bc3c9ba632b2c4947e"}
```



Prometheus scrapes it, Grafana shows it

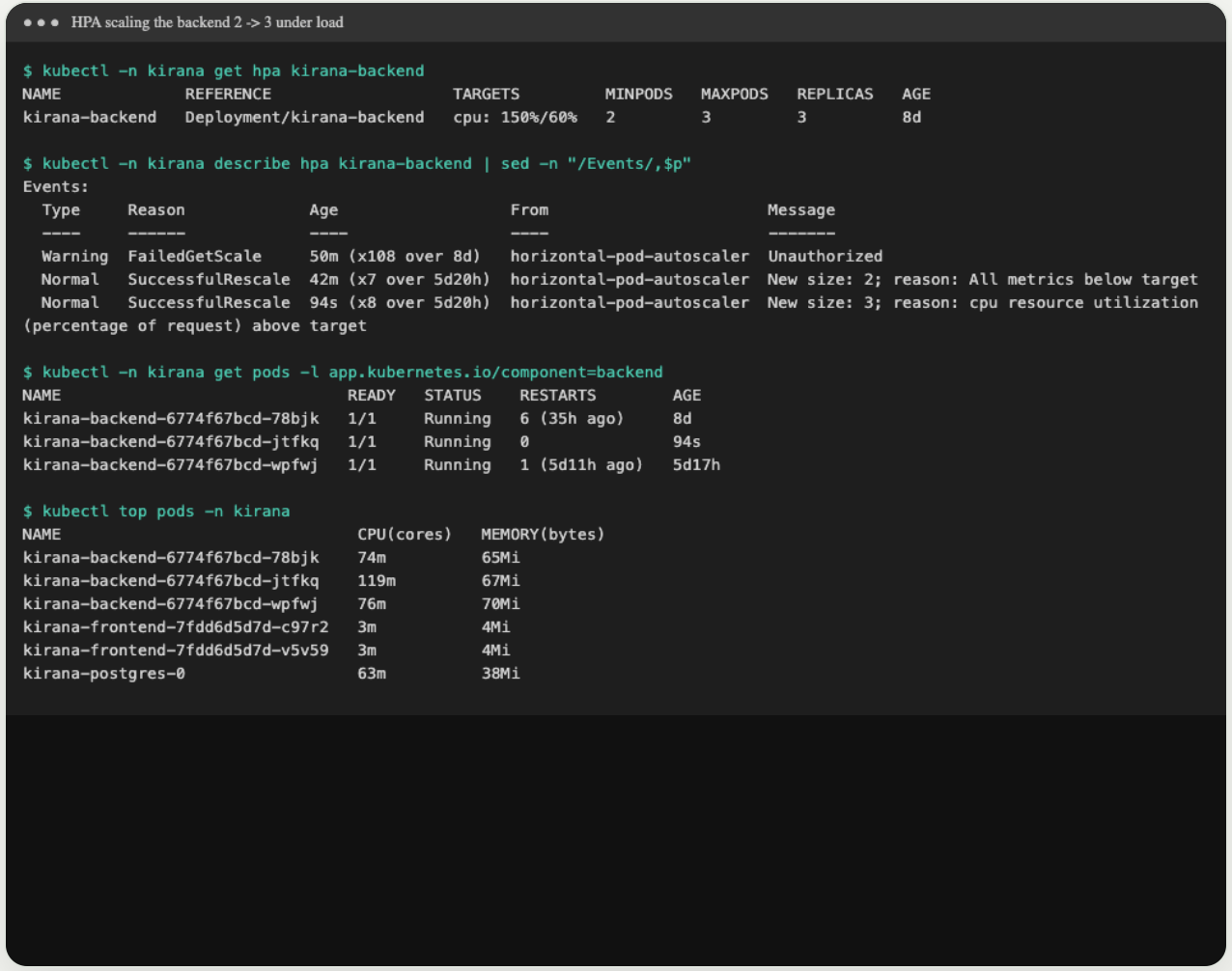




3/3 targets UP · 3 alert rules · dashboard shipped in the chart · gap at 11:27 = Prometheus restart on a memory-starved node

AUTOSCALING

HPA: 2 → 3 backend Pods under load



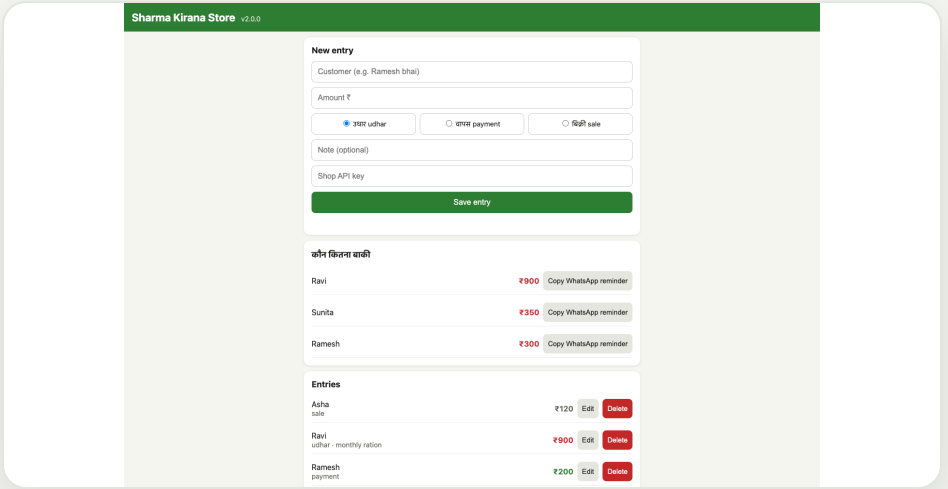
- ~2.5 min of traffic through the Ingress
- CPU **150 %** of request vs **60 %** target
- Event: *"New size: 3; reason: cpu resource utilization above target"*
- Max is 3 on minikube (8 GB laptop), 5 on EKS
- Needs **resource requests** + **metrics-server**

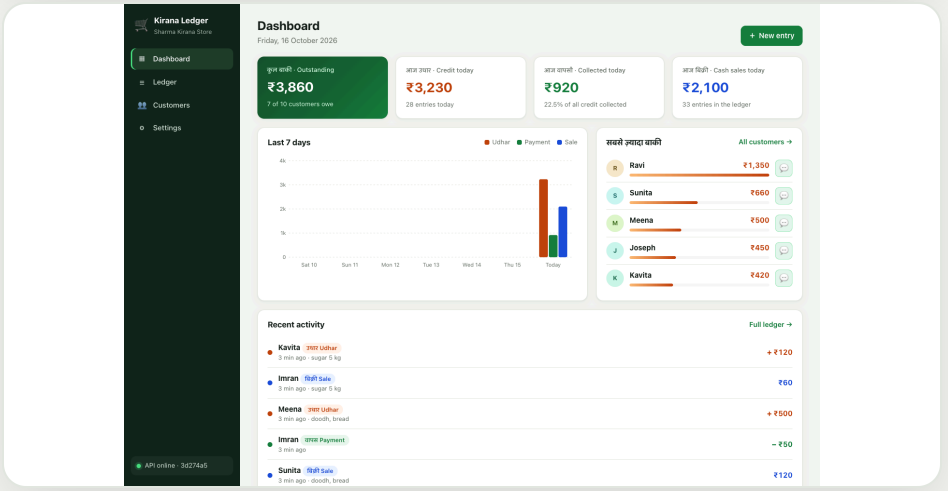
helm/kirana/templates/hpa.yaml

LIVE DEMO · M10

Commit → pipeline → cluster updates itself

```
fb74dbf  release 2.1.0 pushed        → pipeline 13/13 green
c8e01c9  promote it (1-line commit) → Argo CD syncs
         new Pods: exec format error
         (old 2.0.0 Pods keep serving users)
         cause: CI built amd64, Mac minikube is arm64
c722d8a  multi-arch build fix        → 17/17 green
         one :sha tag = amd64 + arm64
41ee491  promote fixed images        → Argo CD rolls
         2.0.0 → 2.1.0 in ~6 min, 0 downtime
         running image = exactly what CI scanned
```





output-gitops-rollout.txt · output-incident-arch.txt — the incident was real, not staged

TROUBLESHOOTING

Two real incidents, two lessons

1 · Postgres CrashLoopBackOff

8 GB node at 419 % CPU; `pg_isready` with a 1 s liveness timeout timed out → kubelet kept killing a *healthy* database → every API Pod 0/1.

Fix: patient DB liveness (30 s delay, 5 s timeout, 6 failures), configurable probe timeouts, API boots without DB, HPA max 3.

Lesson: liveness restarts; readiness only removes from the Service.

2 · exec format error

GitOps rollout to the Apple Silicon node crash-looped: images were `linux/amd64`, node is `arm64`.

Fix: build each tier on native amd64 + arm64 runners, scan all 4, publish one multi-arch tag per SHA.

Lesson: the rolling update protected users — new Pods never got Ready, old ones kept serving.

WRAP-UP

What I'd do next in production

- Amazon RDS instead of Postgres in the cluster (backups, failover)
- CI writes the promotion commit itself (Argo CD Image Updater)
- Sealed Secrets / External Secrets for the API key
- cert-manager TLS on the Ingress
- Remote Terraform state (S3 + DynamoDB lock)
- Alertmanager → Slack/WhatsApp
- Load test in CI with SLO checks (p95 < 300 ms)
- Voice input for shopkeepers (Hindi + English)

धन्यवाद · Thank you

Questions? · `session-21-final-devops-project/`